TamboUI separates physical input (keys, mouse clicks) from semantic actions. Instead of checking for specific keys everywhere, you define bindings that map inputs to actions like "moveUp" or "delete".

Predefined Binding Sets

Several binding presets are available:

```
Bindings standard = BindingSets.standard();  // Arrow keys, Enter, Escape
Bindings vim = BindingSets.vim();            // hjkl navigation, Ctrl+u/d
Bindings emacs = BindingSets.emacs();        // Ctrl+n/p/f/b navigation
```

Matching Actions in Events

Events carry binding information. Use semantic checks instead of hardcoding keys:

```
// Semantic - works with any binding set
if (event.isUp()) { }
if (event.isSelect()) { }
if (event.matches("delete")) { }

// Low-level - when you need specific keys
if (event.isChar('x')) { }
if (event.isKey(KeyCode.F1)) { }
```

The `is*()` methods delegate to the current bindings. With vim bindings, `isUp()` matches Up arrow, `k`, or `Ctrl+p`. With standard bindings, it only matches the Up arrow.

Key Triggers

`KeyTrigger` defines what input triggers an action:

```
KeyTrigger.key(KeyCode.UP);          // Arrow key
KeyTrigger.ch('j');                  // Character
KeyTrigger.chIgnoreCase('j');        // j or J
KeyTrigger.ctrl('u');                // Ctrl+U
KeyTrigger.alt('x');                 // Alt+X
```

Terminal Compatibility

Ctrl key bindings have significant limitations due to how terminals encode keyboard input.

When you press a key combination like Ctrl+A, the terminal does not send "Ctrl" + "A" separately. Instead, it sends a single ASCII control character (byte value 1 for Ctrl+A). This encoding dates back to the 1970s and causes several problems:

- **Ctrl+letter (a-z)**: Works, but case information is lost. Ctrl+a and Ctrl+Shift+A send the same byte.
- **Ctrl+number**: Unreliable. Ctrl+2, Ctrl+@, and Ctrl+Space all send the same byte (0). The original key cannot be determined.
- **Ctrl+Alt+key**: Inconsistent across terminals.

Alt key bindings are more reliable because terminals send ESC followed by the actual character:

- Alt+a sends `ESC` + `'a'` - we know it was lowercase 'a'
- Alt+A sends `ESC` + `'A'` - we know it was uppercase 'A' (shifted)
- Alt+2 sends `ESC` + `'2'` - we know it was '2'

Recommendation for custom bindings:

Modifier	Reliability	Notes
`Alt+letter`	Excellent	Preserves case, works consistently

Modifier	Reliability	Notes
`Alt+number`	Excellent	Fully distinguishable
`Ctrl+letter`	Good	Works but loses shift information
`Ctrl+number`	Poor	Avoid - ambiguous with other combinations

For maximum compatibility, prefer `Alt+key` bindings for custom actions. Use `Ctrl+key` only for well-established conventions (Ctrl+C, Ctrl+S, Ctrl+Z, etc.).

Mouse Triggers

```
MouseTrigger.click();                  // Left click
MouseTrigger.rightClick();             // Right click
MouseTrigger.ctrlClick();              // Ctrl+click
MouseTrigger.scrollUp();               // Scroll wheel
MouseTrigger.drag(MouseButton.LEFT);   // Dragging
```

Custom Bindings

Start from a preset and customize:

```
Bindings custom = BindingSets.standard()
    .toBuilder()
    .bind(KeyTrigger.ch('d'), "delete")
    .bind(KeyTrigger.key(KeyCode.DELETE), "delete")
    .bind(KeyTrigger.ctrl('s'), "save")
    .bind(MouseTrigger.rightClick(), "contextMenu")
    .build();
```

ActionHandler

For centralized action handling, use `ActionHandler`:

```
ActionHandler actions = new ActionHandler(BindingSets.vim())
    .on(Actions.QUIT, e -> runner.quit())
    .on("save", e -> save())
    .on("delete", e -> deleteSelected());
```

```
// In your event handler:
boolean handleEvent(Event event, TuiRunner runner) {
    ActionHandler actions = new ActionHandler(BindingSets.vim());
    if (actions.dispatch(event)) {
        return true;  // Action was handled
    }
    // Handle other events...
    return false;
}
```

Handlers can optionally receive the action name that triggered them:

```
// Handler with action name (useful when same handler serves multiple actions)
ActionHandler actions = new ActionHandler(bindings)
    .on("red", (event, action) -> setColor(action))
    .on("blue", (event, action) -> setColor(action))
    .on("green", (event, action) -> setColor(action));
```

@OnAction Annotation

> The `@OnAction` annotation works with Toolkit Components. For TuiRunner or immediate mode, use `ActionHandler` instead (see above).

In Toolkit components, annotate methods to handle actions:

```
public class EditorComponent extends Component<EditorComponent> {
    private List<String> lines = new ArrayList<>();
    private int cursor = 0;

    @OnAction(Actions.MOVE_UP)
    void moveCursorUp(Event event) {
        if (cursor > 0) cursor--;
    }

    @OnAction(Actions.MOVE_DOWN)
    void moveCursorDown(Event event) {
        if (cursor < lines.size() - 1) cursor++;
    }

    @OnAction("delete")
    void deleteLine(Event event) {
        if (!lines.isEmpty()) {
            lines.remove(cursor);
        }
    }

    @Override
    protected Element render() {
        return text("Editor");
    }
}
```

The component framework discovers these methods and dispatches events automatically.

Annotation Processing

By default, `@OnAction` methods are discovered at runtime using reflection. For better startup performance and GraalVM native image compatibility, use the annotation processor to generate action handler registrations at compile time.

Gradle (Kotlin DSL)

```
dependencies {
    implementation("dev.tamboui:tamboui-toolkit:0.2.0-SNAPSHOT")
```

```
    annotationProcessor("dev.tamboui:tamboui-processor:0.2.0-SNAPSHOT")
}
```

Gradle (Groovy DSL)

```
dependencies {
    implementation 'dev.tamboui:tamboui-toolkit:0.2.0-SNAPSHOT'
    annotationProcessor 'dev.tamboui:tamboui-processor:0.2.0-SNAPSHOT'
}
```

Maven

```
<dependencies>
    <dependency>
        <groupId>dev.tamboui</groupId>
        <artifactId>tamboui-toolkit</artifactId>
        <version>0.2.0-SNAPSHOT</version>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.13.0</version>
            <configuration>
                <annotationProcessorPaths>
                    <path>
                        <groupId>dev.tamboui</groupId>
                        <artifactId>tamboui-processor</artifactId>
                        <version>0.2.0-SNAPSHOT</version>
                    </path>
                </annotationProcessorPaths>
            </configuration>
        </plugin>
    </plugins>
</build>
```

The processor generates a `_ActionHandlerRegistrar` class for each component with `@OnAction` methods. These registrars are discovered via `ServiceLoader` and register action handlers without reflection.

Loading Bindings from Files

Bindings can be externalized to properties files:

```
// From classpath
Bindings bindings = BindingSets.loadResource("/my-bindings.properties");

// From filesystem
Bindings bindingsFromFile = BindingSets.load(Paths.get("~/.config/myapp/bindings.p
```

Property file format:

```
# Navigation
moveUp = Up, k
moveDown = Down, j
pageUp = PageUp, Ctrl+u
pageDown = PageDown, Ctrl+d

# Actions
select = Enter, Space
cancel = Escape
delete = d, Delete

# Mouse
click = Mouse.Left.Press
contextMenu = Mouse.Right.Press
```

Multiple triggers for the same action are comma-separated. Modifiers use `Ctrl+`, `Alt+`, `Shift+` prefixes.

Standard Actions

The `Actions` class defines common action names:

Action	Description
`MOVE_UP`, `MOVE_DOWN`, `MOVE_LEFT`, `MOVE_RIGHT`	Navigation
`PAGE_UP`, `PAGE_DOWN`, `HOME`, `END`	Page navigation
`SELECT`, `CONFIRM`, `CANCEL`	Selection and confirmation
`FOCUS_NEXT`, `FOCUS_PREVIOUS`	Focus navigation (Tab/Shift+Tab)
`DELETE_BACKWARD`, `DELETE_FORWARD`	Text editing
`QUIT`	Application exit

Next Steps

- API Levels - choosing between immediate mode, TuiRunner, and Toolkit
- Application Structure - patterns for larger applications

PREVIOUS

CSS Styling

NEXT

MVC Model